

AeroWMS — Technical Portfolio & Interview Dossier

Enterprise Multi-Tenant Warehouse Management System • Production Architecture

Candidate Name:	Izza Faris	Specialization:	Full Stack / Java Spring Boot & Angular
Candidate Email:	izzafaris.it@gmail.com	GitHub Profile:	github.com/izza-faris
GitHub Repo:	github.com/izza-faris/WMS-SYSTEM	Live Application:	izza-faris.github.io/WMS-SYSTEM
Backend API:	Railway Cloud API (Status: UP)	Tech Stack:	Spring Boot 3.3 • Angular 18 • Mongo DB Atlas

1. Live Project & Demo Access

AeroWMS is fully deployed in a modern decoupled cloud environment with zero-downtime CI/CD:

Service Component	Deployment Infrastructure	Production URL & Health Telemetry
Frontend Web Portal	GitHub Pages (Static SPA)	https://izza-faris.github.io/WMS-SYSTEM/
Backend Cloud REST API	Railway Cloud PaaS (Docker/Java 21)	https://wms-system-production-e062.up.railway.app/api/v1
Service Health Telemetry	Spring Boot Actuator	https://wms-system-production-e062.up.railway.app/api/health ONLINE: UP
Source Code Repository	GitHub Public Repository	https://github.com/izza-faris/WMS-SYSTEM

Multi-Tenant Evaluation Credentials

The cloud database contains pre-seeded multi-tenant workspaces to verify complete tenant isolation:

Role	Tenant Organization	Email / Username	Password	Operational Scope
PLATFORM SUPER ADMIN	AeroWMS Global	admin@aerowms.com	admin123	Global SaaS tenant provisioning, health telemetry
TENANT ADMIN	Apex Retailers Pvt Ltd	admin@apexretailers.com	Apex@123	Full enterprise tenant administration, users, billing
WAREHOUSE MANAGER	Apex - Colombo Central	manager.colombo@apexretailers.com	Manager@123	Branch inventory, transfers, adjustments, approvals
WAREHOUSE STAFF	Apex - Colombo Central	staff.colombo@apexretailers.com	Staff@123	Barcode scanning, put-away, picking, dispatch
TENANT B ADMIN	Zenith Logistics Inc	admin@zenithlogistics.com	Zenith@123	Isolated secondary tenant (zero cross-tenant data leakage)

2 & 3. GitHub Repository & Codebase Architecture

The complete source code is maintained with clear architectural separation of concerns:

```
wms-system/
├── backend/                                # Spring Boot 3.3.3 Enterprise Application (Java 21 LTS)
│   ├── src/main/java/com/wms/
│   │   ├── config/                        # SecurityConfig, JwtAuthenticationFilter, TenantContext, MongoConfig
│   │   ├── controller/                   # REST Controllers (Auth, Inventory, Warehouse, Transfer, Report, etc.)
│   │   ├── dto/                          # Strongly typed request/response transfer schemas
│   │   ├── entity/                       # MongoDB document persistence models (Client, Product, Inventory, etc.)
│   │   ├── exception/                    # Centralized GlobalExceptionHandler & Domain Business Exceptions
│   │   ├── repository/                   # Spring Data Repositories with custom compound index queries
│   │   └── service/                       # Core Domain Services: FEFO engine, StockTransfer, ZXing QR engine
│   └── pom.xml                           # Maven dependency descriptor
├── frontend/                             # Angular 18 Single-Page Application (Standalone Components)
│   ├── src/app/
│   │   ├── components/                   # 18 Independent feature modules (Dashboard, Scanner, Warehouse, etc.)
│   │   ├── guards/                       # AuthGuard and Role-Based route activation policies
│   │   ├── models/                       # TypeScript data interfaces matching backend DTO contracts
│   │   └── services/                     # WmsApiService (HTTP Client with Bearer JWT interceptor)
│   └── package.json                       # NPM dependencies & build scripts
└── .github/workflows/deploy.yml           # Automated GitHub Actions deployment pipeline for GitHub Pages
```

4. Database Entity-Relationship Model

Modeled for high scalability using MongoDB document persistence with relational document scoping:

```
/* MULTI-TENANT ORGANIZATIONAL HIERARCHY */ [CLIENT] 1 — ∞ [BRANCH] 1 — ∞ [WAREHOUSE] 1 — ∞ [ZONE] | | | — ∞
[USER] | — ∞ [USER (Assigned Branch)] | /* 5-TIER SPATIAL STORAGE HIERARCHY */ [WAREHOUSE] 1 — ∞ [ZONE] 1 — ∞
[RACK] 1 — ∞ [SHELF] 1 — ∞ [BIN (Location QR)] | /* INVENTORY, BATCH & FEFO LEDGER */ [PRODUCT] 1 — ∞
[PRODUCT_BATCH (ExpiryDate)] | | | | — 1 — ∞ [INVENTORY (quantity, reservedQuantity)] ————— |
/* TRANSACTION AUDITING & WORKFLOWS */ [STOCK_TRANSACTION] (STOCK_IN, STOCK_OUT, TRANSFER_IN, TRANSFER_OUT,
ADJUSTMENT) [STOCK_TRANSFER] 1 — ∞ [STOCK_TRANSFER_ITEM] (PENDING → APPROVED → DISPATCHED → RECEIVED)
[STOCK_ADJUSTMENT] (Discrepancy quantity, reason: Damage / Shrinkage / Count Error) [AUDIT_LOG] (Immutable security
trail: userEmail, Action, TargetEntity, IP, Timestamp)
```

Collection / Entity	Primary Keys & Foreign References	Enforced Compound Indexes & Constraints
clients	id (PK)	Unique: contactEmail
users	id (PK), clientId, branchId	Unique: email; Index: (clientId, role)
warehouses & bins	id (PK), clientId, shelfId	Unique: (clientId, qrCode), (shelfId, code)
products	id (PK), clientId, categoryId	Unique: (clientId, sku); Index: (clientId, barcode)
product_batches	id (PK), clientId, productId	Index: (clientId, productId, expiryDate ASC) (FEFO speed)
inventory	id (PK), clientId, warehouseId, binId	Unique: (clientId, warehouseId, binId, productId, batchId)
stock_transactions	id (PK), clientId, performedByUserId	Index: (clientId, createdAt DESC)

5. System Architecture & Request Lifecycle

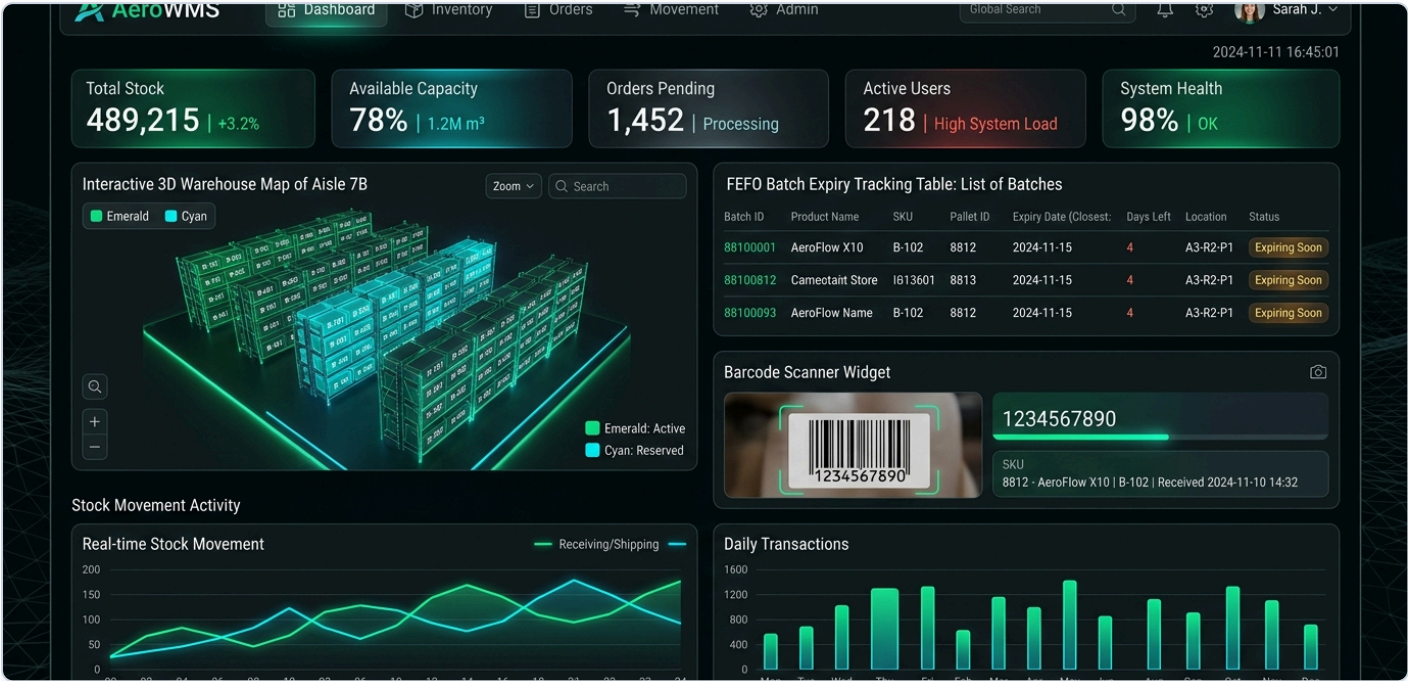
```
[Angular 18 Client] → (HTTPS + Bearer JWT) | ▼ [Spring Security 6 Gateway] |— CORS Preflight Filter |—
JwtAuthenticationFilter: Validates token signature & expiration | |— Claims: Extract (email, userId, clientId,
branchId, role) | |— TenantContext.set(tenantInfo) (ThreadLocal storage) | |—
SecurityContextHolder.setAuthentication(UsernamePasswordToken) | ▼ [Domain Services Layer] |— TenantSecurityService:
Asserts (entity.clientId == currentTenantId) |— InventoryService: FEFO logic, atomic balance updates
(@Transactional) |— StockTransferService: 4-stage transfer state machine & stock reservation |— QrBarcodeService:
Google ZXing BitMatrix Base64 image generation |— ReportService: Apache POI Excel (.xlsx) & OpenPDF generator | ▼
[Persistence Tier: MongoDB Atlas] |— Scoped queries: findByClientIdAndWarehouseId(...) | ▼ [Finally Filter Block]
|— TenantContext.clear() (Guarantees zero ThreadLocal memory leaks)
```

6. REST API Documentation Catalog

Controller	Method & Route	Request Body / Params	Response Contract & Status
AuthController	POST /api/v1/auth/login	{ email, password }	{ token, role, clientId, fullName } (200)
AuthController	POST /api/v1/auth/register-client	{ orgName, email, password, phone }	Provisions tenant, admin & default branch (201)
InventoryController	GET /api/v1/inventory/balances	? warehouseId=&productId=	List<InventoryBalanceDto> (200)
InventoryController	GET .../fefo-recommendations	? productId=&warehouseId=	List<FefoBatchRecommendationDto> (200)
InventoryController	POST /api/v1/inventory/stock-in	{ warehouseId, binId, prodId, batch, qty }	Allocates bin, writes STOCK_IN transaction
InventoryController	POST /api/v1/inventory/stock-out	{ warehouseId, prodId, qty, batchId }	Applies FEFO priority, validates negative stock
WarehouseController	GET /api/v1/warehouses/{id}/tree	Path: warehouseId	Returns full 5-tier spatial tree (Warehouse → Bin)
WarehouseController	POST .../shelves/{id}/bins	{ code, capacityCubicMeters }	Auto-generates location code & Base64 QR code
TransferController	POST /api/v1/transfers	{ srcWhId, destWhId, items: [...] }	Reserves source stock, creates transfer (PENDING)
TransferController	PUT /api/v1/transfers/{id}/dispatch	Path: transferId	Deducts reserved stock, status: DISPATCHED
TransferController	PUT /api/v1/transfers/{id}/receive	Path: transferId	Adds stock to dest bins, status: RECEIVED
ProductController	GET /api/v1/products/scan/{code}	Path: code (Barcode or SKU)	Fast-lookup for optical camera & laser scanner gun
ReportController	GET /api/v1/reports/inventory/excel	None (Reads tenant context)	Binary formatted Excel spreadsheet (.xlsx)
ReportController	GET /api/v1/reports/inventory/pdf	None (Reads tenant context)	Binary formatted PDF valuation document (.pdf)

7. Major Modules & Visual Walkthrough

High-resolution executive UI rendering showing live analytics, FEFO batch tracking, and optical scanner modal:



1. Executive Analytics & Telemetry

Real-time KPI cards for aggregate stock units, cubic capacity utilization, active users, and Chart.js inventory throughput curves.

2. Optical Camera & Laser Scanner

Dual camera viewfinder with animated laser guide, Web Audio API chime, high-speed laser scanner gun input, and instant sticker printing.

3. 5-Tier Spatial Hierarchy Tree

Visual mapping of Warehouse → Zone → Rack → Shelf → Bin. Generates standardized QR codes (e.g. WH01-ZA-R01-S02-B03).

4. Smart FEFO Inbound & Outbound

Captures batch expiry dates on Stock In. Enforces earliest-expiry recommendations on Stock Out to minimize product spoilage.

5. Inter-Warehouse Transfers

4-stage state machine (PENDING → APPROVED → DISPATCHED → RECEIVED) with inventory reservation preventing stock race conditions.

6. Audit Logs & POI/PDF Reporting

Complete immutable user audit trails and dynamic server-side Excel/PDF report generation powered by Apache POI and OpenPDF.

8. 2–4 Minute Live Demo Video Storyboard & Script

Timestamp	Screen & Action	Spoken Presentation Script / Key Points
0:00 – 0:35	Landing & Login Open portal, select admin@apexretailers.com	"Hello, I am Izza Faris. Today I am demonstrating AeroWMS, an enterprise multi-tenant Warehouse Management System built with Spring Boot 3 and Angular 18. Each tenant is strictly isolated in memory and database using ThreadLocal context."
0:35 – 1:15	Dashboard & Hierarchy Show KPIs, expand warehouse down to Bin node	"On the dashboard, we monitor real-time stock levels and turnover. In our Warehouses module, AeroWMS models physical storage down 5 levels: Warehouse, Zone, Rack, Shelf, and Bin. Each bin automatically receives a standardized QR code like WH01-ZA-R01-S02-B03 via Google ZXing."
1:15 – 1:55	Optical Scanner & Price Scan barcode, trigger audio chime, show label	"Our optical scanner supports both camera streams with an animated laser overlay and physical USB laser scanner guns. Decodes trigger a synthesized Web Audio API chime, display live pricing, and allow instant label printing."
1:55 – 2:40	FEFO Stock In / Out Enter batch expiry, show FEFO priority ordering	"AeroWMS enforces FEFO—First-Expired-First-Out. When goods arrive, we record batch numbers and expiry dates. When dispatching, our algorithm automatically prioritizes the earliest expiring batch to eliminate inventory loss."
2:40 – 3:15	Transfers & Reports Demonstrate multi-item transfer state & download Excel	"For inter-warehouse logistics, transfers automatically reserve source inventory to prevent race conditions during transit. Every action is written to immutable audit logs and exportable to Excel and PDF via Apache POI and OpenPDF."
3:15 – 3:30	Conclusion Summary of cloud stack	"AeroWMS is deployed live to production on Railway and GitHub Pages with automated CI/CD. Thank you for your time."

9. Complete Technology Stack & Libraries

Domain	Technology / Library	Version	Architectural Purpose
Backend Runtime	Java & Spring Boot	Java 21 LTS / Spring Boot 3.3.3	Enterprise application framework with virtual threads
Security	Spring Security & JWT	Spring Security 6 / JWT 0.12.6	Stateless JWT token authentication, BCrypt, RBAC
Database & ORM	Spring Data MongoDB	MongoDB Driver 5.x / Atlas	Document persistence with compound index queries
Barcode / QR	Google ZXing	ZXing 3.5.3 (core & javase)	QR Code & Code-128 BitMatrix Base64 image engine
Reporting Engines	Apache POI & OpenPDF	POI 5.2.5 / OpenPDF 1.3.40	Binary streaming export of formatted Excel & PDF ledgers
Frontend Framework	Angular	Angular 18.2 (Standalone)	Component-based SPA, Signals reactivity, typed forms
Optical Decoding	html5-qrcode	2.3.8	HTML5 canvas/video real-time camera barcode decoder
Audio Telemetry	Browser Web Audio API	Native HTML5 API	1760 Hz sine wave tone synthesis for scan verification
Data Visualization	Chart.js	4.4.4	Interactive inventory trend & stock movement analytics
DevOps & Cloud	GitHub Actions & Railway	CI/CD v4	Automated testing, compilation, and cloud deployment

11. Setup, Limitations & Future Roadmap

Local Setup Instructions:

1. Backend: `cd backend && mvn clean spring-boot:run` (Starts on port 8080)
2. Frontend: `cd frontend && npm install && npm start` (Opens on port 4200)
3. Windows 1-Click: Run `run-wms.bat` to launch both services simultaneously.

Known Technical Limitations (Senior Self-Critique):

- *Logical vs. Physical Database Isolation*: Uses scoped `clientId` partitioning; regulated banking clients may require dedicated database clusters.
- *Push Notifications*: Currently polls on navigation; integrating Spring STOMP/WebSockets will provide instant push notifications.
- *Camera API over LAN*: Modern mobile browsers require HTTPS for `getUserMedia`; local network testing requires self-signed SSL certs.

Future Roadmap:

- **IoT RFID Gate Microservice**: MQTT/AMQP listener to check in whole pallets automatically at warehouse dock doors.
- **AI Demand Forecasting**: Python Prophet/ARIMA sidecar service analyzing stock history for predictive replenishment.
- **Carrier Integrations**: Direct API webhooks for DHL, FedEx, and UPS automated shipping label generation.

14. Comprehensive Technical Interview Defense Guide

Q1. Why did you choose this technology stack?

Spring Boot 3 + Java 21 was chosen for enterprise-grade type safety, mature dependency injection, and robust security via Spring Security 6. Java 21 virtual threads support high-throughput concurrent inventory operations. **MongoDB Atlas** was selected because warehouse stock involves heterogeneous products (some have serial numbers, batch expiry, or volumetric attributes) without needing relational migration locks, while compound indexes ensure sub-millisecond lookups on (`clientId`, `warehouseId`, `productId`). **Angular 18** provides a structured foundation with Signals and Reactive Forms for data-heavy warehouse tables.

Q2. How does Multi-Tenancy work under the hood?

AeroWMS uses **Logical Document Partitioning with ThreadLocal Isolation**. Every incoming HTTP request carries a signed JWT containing `clientId` and `role`. In `JwtAuthenticationFilter`, the claims are extracted and set into a `ThreadLocal<TenantInfo>` context. Whenever domain services query data, `TenantSecurityService` retrieves the authenticated `clientId` and enforces scoped queries (e.g. `findByClientIdAndWarehouseId`). A finally block ensures `TenantContext.clear()` is executed to prevent thread pool contamination in Tomcat.

Q3. How is JWT implemented?

Implemented with `io.jsonwebtoken (jjwt 0.12.6)`. Tokens are cryptographically signed with HMAC-SHA256 using a 256-bit secret with a 24-hour expiration. The payload contains claims: `sub` (email), `userId`, `clientId`, `branchId`, and `role`. The Angular HTTP Interceptor attaches the token as a Bearer header. In Spring, `JwtAuthenticationFilter` validates the cryptographic signature and registers a `UsernamePasswordAuthenticationToken` in Spring's `SecurityContextHolder`.

Q4. How is the 5-tier Warehouse Hierarchy designed?

Modeled as a 5-tier spatial tree: **Warehouse → Zone → Rack → Shelf → Bin**. Bins represent the smallest physical unit where inventory resides and receive a standardized hierarchical location code: `[WarehouseCode] - [ZoneCode] - [RackCode] - [ShelfCode] - [BinCode]` (e.g., `WH01-ZA-R01-S02-B03`). Google ZXing dynamically renders this code into a Base64 QR code so warehouse operators can scan the bin label during put-away or pick operations.

Q5. How does the FEFO (First-Expired-First-Out) logic work?

When goods are received during `StockIn`, the manufacturing and expiration dates are saved in the `ProductBatch` collection. During a `StockOut` request, `InventoryService` queries available batches sorted ascending by `expiryDate` where `availableQuantity > 0`. Batches closest to expiration are automatically recommended and allocated first, preventing stock spoilage in perishable food and pharmaceutical distribution.

Q6. How are Stock Transfers processed?

Stock transfers follow a 4-state finite state machine:

- PENDING:** Validates cross-warehouse tenant ownership. Stock is immediately **reserved** (`reservedQuantity += qty`) at the source warehouse to eliminate overselling race conditions.
- APPROVED:** Authorized by a warehouse manager.
- DISPATCHED:** Physical inventory is decremented (`quantity -= qty`, `reservedQuantity -= qty`) and a `TRANSFER_OUT` transaction is recorded.
- RECEIVED:** Destination warehouse staff scan the goods into destination bins (`quantity += qty`) and a `TRANSFER_IN` transaction is logged.

Q7. How does QR & Barcode scanning work?

Camera Scanning: Angular uses `html5-qrcode` to capture video stream frames via `getUserMedia`, processing canvas frames in real time to decode 1D barcodes and 2D QR codes.

Laser Scanner Gun: Handheld USB/Bluetooth scanners operate as keyboard wedge devices. The scanner component listens to `keyup.enter` on an auto-refocused text field for hands-free continuous scanning.

Audio Feedback: Emulates industrial Honeywell scanners by using the browser's native `AudioContext` to synthesize a 1760 Hz sine wave tone on successful decode.

Label Generation: Backend `QrBarcodeService` uses ZXing to generate Code-128 barcode `BitMatrix` PNGs for printable price sticker labels.

Q8. What security measures did you implement?

- **BCrypt Password Hashing:** Passwords encrypted with strength factor 10.
- **Stateless JWT Authentication:** Validated cryptographically on every request.
- **Injection Prevention:** MongoDB parameterized query builders prevent NoSQL injection; Angular's DomSanitizer prevents XSS.
- **Tenant Access Control:** TenantSecurityService enforces strict tenant ownership checks.
- **Immutable Audit Trail:** Administrative and operational actions write to an append-only AuditLog collection.

Q9. What was the most challenging technical challenge in this project?

Challenge: Preventing inventory race conditions and negative balances during concurrent stock-out and inter-warehouse transfers in a multi-tenant environment.

Solution: Implemented atomic stock balance verification within Spring's `@Transactional` boundaries, paired with stock reservation states during inter-warehouse transfers (`reservedQuantity`). Calculated effective available stock (`quantity - reservedQuantity`) before fulfilling any dispatch, backed by compound unique index constraints in MongoDB to prevent duplicate inventory records.